

6.2 链 表

到目前为止,已经大量地使用过了数组及其不定长版本——vector,使用的方法大都是随机存取和往末尾添加/删除元素。但有时也需要向数组中插入元素,下面便是一例。

例题 6-4 破损的键盘(又名:悲剧文本)(Broken Keyboard(a.k.a. Beiju Text), UVa 11988)

你有一个破损的键盘。键盘上的所有键都可以正常工作,但有时 Home 键或者 End 键会自动按下。你并不知道键盘存在这一问题,而是专心地打稿子,甚至连显示器都没打开。当你打开显示器之后,展现在你面前的是一段悲剧的文本。你的任务是在打开显示器之前计算出这段悲剧文本。

输入包含多组数据。每组数据占一行,包含不超过 100000 个字母、下划线、字符 “[” 或者 “]”。其中字符 “[” 表示 Home 键,“]” 表示 End 键。输入结束标志为文件结束符 (EOF)。输入文件不超过 5MB。对于每组数据,输出一行,即屏幕上的悲剧文本。

样例输入:

```
This_is_a_[Beiju]_text
[[]][[]]Happy_Birthday_to_Tsinghua_University
```

样例输出:

```
BeijuThis_is_a__text
Happy_Birthday_to_Tsinghua_University
```

【分析】

最简单的想法便是用数组来保存这段文本,然后用一个变量 pos 保存“光标位置”。这样,输入一个字符相当于在数组中插入一个字符(需要先把后面的字符全部右移,给新字符腾出位置)。

很可惜,这样的代码会超时。为什么?因为每输入一个字符都可能会引起大量字符移动。在极端情况下,例如,2500000 个 a 和 “[” 交替出现,则一共需要 $0+1+2+\cdots+2499999=6*10^{12}$ 次字符移动。

解决方案是采用链表(linked list)。每输入一个字符就把它存起来,设输入字符串是 $s[1\sim n]$,则可以用 $next[i]$ 表示在当前显示屏中 $s[i]$ 右边的字符编号(即在 s 中的下标)^①。

提示 6-3: 在数组中频繁移动元素是很低效的,如有可能,可以使用链表。

为了方便起见,假设字符串 s 的最前面还有一个虚拟的 $s[0]$,则 $next[0]$ 就可以表示显示屏中最左边的字符。再用一个变量 cur 表示光标位置:即当前光标位于 $s[cur]$ 的右边。cur=0 说明光标位于“虚拟字符” $s[0]$ 的右边,即显示屏的最左边。

提示 6-4: 为了方便起见,常常在链表的第一个元素之前放一个虚拟结点。

^① 读者可能在其他数据结构书中见过基于指针的链表实现方式,但是链表并不一定要用指针。

为了移动光标，还需要用一个变量 `last` 表示显示屏的最后一个字符是 `s[last]`。代码如下：

```
#include<cstdio>
#include<cstring>
const int maxn = 100000 + 5;
int last, cur, next[maxn]; //光标位于 cur 号字符的后面
char s[maxn];

int main() {
    while(scanf("%s", s+1) == 1) {
        int n = strlen(s+1); //输入保存在 s[1], s[2]...中
        last = cur = 0;
        next[0] = 0;
        for(int i = 1; i <= n; i++){
            char ch = s[i];
            if(ch == '[') cur = 0;
            else if(ch == ']') cur = last;
            else {
                next[i] = next[cur];
                next[cur] = i;
                if(cur == last) last = i; //更新“最后一个字符”编号
                cur = i; //移动光标
            }
        }
        for(int i = next[0]; i != 0; i = next[i])
            printf("%c", s[i]);
        printf("\n");
    }
    return 0;
}
```

例题 6-5 移动盒子 (Boxes in a Line, UVa 12657)

你有一行盒子，从左到右依次编号为 $1, 2, 3, \dots, n$ 。可以执行以下 4 种指令：

- 1 XY 表示把盒子 X 移动到盒子 Y 左边（如果 X 已经在 Y 的左边则忽略此指令）。
- 2 XY 表示把盒子 X 移动到盒子 Y 右边（如果 X 已经在 Y 的右边则忽略此指令）。
- 3 XY 表示交换盒子 X 和 Y 的位置。
- 4 表示反转整条链。

指令保证合法，即 $X \neq Y$ 。例如，当 $n=6$ 时在初始状态下执行 114 后，盒子序列为 231456。接下来执行 235，盒子序列变成 214536。再执行 316，得到 264531。最终执行 4，得到 135462。

输入包含不超过 10 组数据，每组数据第一行为盒子个数 n 和指令条数 m ($1 \leq n, m \leq$

100000)，以下 m 行每行包含一条指令。每组数据输出一行，即所有奇数位置的盒子编号之和。位置从左到右编号为 $1 \sim n$ 。

样例输入：

```
6 4
1 1 4
2 3 5
3 1 6
4
6 3
1 1 4
2 3 5
3 1 6
100000 1
4
```

样例输出：

```
Case 1: 12
Case 2: 9
Case 3: 2500050000
```

【分析】

根据前面的经验，如果用数组来保存盒子，肯定会超时，但如果像例题 6-4 那样只保存一个 next 值，似乎又不够，怎么办？

解决方法是采用双向链表（doubly linked list）：用 $\text{left}[i]$ 和 $\text{right}[i]$ 分别表示编号为 i 的盒子左边和右边的盒子编号（如果是 0，表示不存在），则下面的过程可以让两个结点相互连接：

```
void link(int L, int R) {
    right[L] = R; left[R] = L;
}
```

提示 6-5：在双向链表这样的复杂链式结构中，往往会编写一些辅助函数用来设置链接关系。

有了这个代码，可以先记录好操作之前 X 和 Y 两边的结点，然后用 link 函数按照某种顺序把它们连起来。操作 4 比较特殊，为了避免一次修改所有元素的指针，此处增加一个标记 inv ，表示有没有执行过操作 4（如果 $\text{inv}=1$ 时再执行一次操作 4，则 inv 变为 0）。这样，当 op 为 1 和 2 且 $\text{inv}=1$ 时，只需把 op 变成 $3-\text{op}$ （注意操作 3 不受 inv 影响）即可。最终输出时要根据 inv 的值进行不同处理。

提示 6-6：如果数据结构上的某一个操作很耗时，有时可以用加标记的方式处理，而不需要真的执行那个操作。但同时，该数据结构的所有其他操作都要考虑这个标记。

下面的核心代码里还有一些可以借鉴的细节处理，请读者仔细阅读：

```

int main() {
    int m, kase = 0;
    while (scanf("%d%d", &n, &m) == 2) {
        for (int i = 1; i <= n; i++) {
            left[i] = i-1;
            right[i] = (i+1) % (n+1);
        }
        right[0] = 1; left[0] = n;
        int op, X, Y, inv = 0;

        while (m--) {
            scanf("%d", &op);
            if (op == 4) inv = !inv;
            else {
                scanf("%d%d", &X, &Y);
                if (op == 3 && right[Y] == X) swap(X, Y);
                if (op != 3 && inv) op = 3 - op;
                if (op == 1 && X == left[Y]) continue;
                if (op == 2 && X == right[Y]) continue;

                int LX = left[X], RX = right[X], LY = left[Y], RY = right[Y];
                if (op == 1) {
                    link(LX, RX); link(LY, X); link(X, Y);
                }
                else if (op == 2) {
                    link(LX, RX); link(Y, X); link(X, RY);
                }
                else if (op == 3) {
                    if (right[X] == Y) { link(LX, Y); link(Y, X); link(X, RY); }
                    else { link(LX, Y); link(Y, RX); link(LY, X); link(X, RY); }
                }
            }
        }

        int b = 0;
        long long ans = 0;
        for (int i = 1; i <= n; i++) {
            b = right[b];
            if (i % 2 == 1) ans += b;
        }
        if (inv && n % 2 == 0) ans = (long long)n*(n+1)/2 - ans;
        printf("Case %d: %lld\n", ++kase, ans);
    }
}

```

```
    }  
    return 0;  
}
```

如果读者曾独立编写过上面的程序，可能会花费较长的时间进行调试。又或者，自以为正确的程序提交到 UVa 上之后却得到 WA 甚至 RE 或者 TLE。在链式结构中，这样的情况是时常发生的，我们需要具备一定的调试和测试能力。

提示 6-7：复杂的链式数据结构往往比较容易写错。在包含多道题目的算法竞赛中，这一特点可以是选题的依据之一。

简单地说，测试的任务就是检查一份代码是否正确。如果找到了错误，最好还能提供一个让它错误的数据。有了错误数据之后，接下来的任务便是调试：看看程序为什么是错的。如果找到了错误，最好把它改对——至少对于刚才的错误数据能得到正确的结果。改对一组数据之后，可能还有其他错误，因此需要进一步测试；即使以前曾经正确的数据，也可能因为多次改动之后反而变错了，需要再次调试。总之，在编码结束后，为了确保程序的正确性，测试和调试往往要交替进行。

提示 6-8：测试的任务就是检查一份代码是否正确。如果找到了错误，最好还能提供一个让它出错的数据；调试的任务是找到错误原因并改正。改正一个错误之后有可能引入新的错误，因此调试和测试往往要交替进行。

如何测试上述代码的正确性呢？一个行之有效的方法是：再找一份完成同样功能的代码与之对比。对于本题来说，可以先写一个基于数组的版本。虽然这个版本会很慢，但正确性比较容易保证。接下来编写一个数据生成器（在第 5 章中曾介绍过这一技巧），并且反复执行下面的操作：生成随机数据，分别执行两个程序，比较它们的结果（俗称“对拍”）。合理地使用操作系统提供的脚本功能，可以自动完成对比测试，具体方法请读者参见附录 A。

提示 6-9：测试数据结构程序的常用方法是对拍：写一个功能相同但速度较慢的简易版本，再写一个数据生成器，不停对比快慢两个程序的输出。简易版本的代码越简单越好，因为重点不在效率，而在正确性。

如果发现让两个程序答案不一致的数据，最好别急着对它进行调试。可以尝试着减小数据生成器中的 n 和 m ，试图找到一组尽量简单的错误数据。一般来说，数据越简单，越容易调试。如果发现只有很大的数据才会出错，通常意味着程序在处理极限数据方面有问题，例如，`is_prime` 中遇到了“过大的 n ”，或者数组开得不够大等。这些都是很实用的技巧，建议读者多多积累。

提示 6-10：数据的复杂性会大大影响调试的难度，因此在找到让程序出错的数据之后最好别急着调试，而应尝试简化数据，或者直接用更小的参数调用数据生成器，以找到更简单的错误数据。

“对拍”也是命题者采用的常用技巧——为了保证官方测试数据的正确性，命题者通